

RiskRadar — Model & Feature Contract (IRE ↔ Chidera)

This is the interface Chidera's backend calls to get an ML fraud probability for a transaction. Anything on this page is a breaking change if altered without both of us agreeing first (per PRD section 11.3, "API contract rule").

1. Feature calculation

Module: `ml/features.py` **Do not reimplement this logic in FastAPI.** Import and call it directly, or port it line-for-line if the backend is a separate service — either way, any change to feature logic happens in this one file and both sides stay in sync.

Input: `Transaction`

```
Transaction(
    transaction_reference: str,
    customer_token: str,
    amount_minor: int,          # kobo, per PRD section 8.3
    occurred_at: datetime,
    channel: str,               # MOBILE_APP, WEB, USSD, POS, ATM, AGENT, BRANCH
    instrument: str,            # TRANSFER, CARD, CASH, WALLET
    payment_rail: str,          # NIP, NEFT, RTGS, CARD_SCHEME, INTERNAL
    device_id: str | None,
    location: str | None,       # state or LGA name, e.g. "Lagos", "Lekki", "Zamfara"
)
```

State: `CustomerState`

Rolling per-customer history needed to compute behavioural features. In the notebooks/simulator this is an in-memory object per customer. **In the live backend, this must be backed by Postgres** — Chidera owns loading a customer's recent transaction history into a `CustomerState` instance before calling `compute_features`, and persisting the update after.

Output: feature dict

`compute_features(txn, state)` returns a dict with exactly the 24 keys in `FEATURE_NAMES` (see `ml/features.py`). Every value is a number; `-1` means "does not apply" (e.g. no prior transaction yet, no device_id given) rather than 0 or null — treat `-1` as a real sentinel, not a missing-value bug.

2. Model output contract

Per PRD section 9.6, the model's output to the backend is small and stable:

```
{
    "fraud_probability": 0.82,          # float, 0-1
    "model_version": "gbm-v1",         # string, bump on every retrain
    "top_contributing_features": [      # optional, top 3 by importance for this prediction
        "is_ato_risk", "amount_ngn", "spending_deviation_score"
```

```
]
}
```

Current status: `model_version = "gbm-v1"` corresponds to `models/fraud_model_v1.pkl` (actually a `HistGradientBoostingClassifier`, trained in `notebooks/05_gbm_baseline.ipynb`). Load it with `joblib.load()`, call `.predict_proba(X)[: , 1]` for `fraud_probability`, where `X` is a `DataFrame/array` built from `compute_features()` output in the exact order of `FEATURE_NAMES`.

`top_contributing_features` is not yet implemented — would use `model.feature_importances_` combined with the specific feature values for that transaction. Flagged as follow-up work, not blocking initial integration.

3. Known limitations Chidera's decision policy must account for

- **No usable single classification threshold exists yet.** At any threshold that preserves useful recall (≤ 0.5), false-positive burden is ~770-813 per 1,000 legitimate transactions — far too high for a raw ML-only cutoff. The ML probability **must** be combined with rule findings in the decision policy (PRD section 8.6-8.7), not used alone to gate ALLOW/REVIEW/HOLD. See `docs/updates/ml-progress-update-3.md` for the full threshold sweep.
- **Rules currently live as model features, not separate rule outputs.** `is_high_risk_state`, `is_high_risk_lga`, `is_ato_risk`, and `geospatial_velocity_anomaly` are computed inside `compute_features()` and fed into the model. If the decision policy needs these as independent, separately-inspectable rule findings (per PRD section 8.6's "rules and ML stay separate"), they can also be read directly off the feature dict BEFORE passing it to the model - the same values, just also surfaced to the rule engine. No duplicate computation needed.
- **Not yet calibrated.** `fraud_probability` is the model's raw output; it has not yet been checked for "does a 0.7 actually mean ~70% of the time it's fraud" (PRD glossary: calibration). Treat it as a ranking signal for now, not a literal probability, until calibration work (tracked as a pending task) is complete.

4. Versioning

Bump `model_version` on every retrain (`gbm-v1`, `gbm-v2`, ...) and note the change in `docs/updates/`. Store `model_version` alongside every assessment (PRD `model_versions` table, section 11.1) so any result can be traced back to exactly which model produced it.